

Step 1: Data Preparation & Model Loading

```
import torch
import torchvision.models as models
import torchvision.transforms as transforms
from torchvision.datasets import CIFAR10
from torch.utils.data import DataLoader, random_split
from PIL import Image
import requests
from io import BytesIO
import torch.nn as nn
import torch.optim as optim

# Load the pre-trained VGG16 model
model = models.vgg16(pretrained=True)

# Modify the classifier to fit CIFAR-10
model.classifier[6] = nn.Linear(4096, 10)

# Move the model to GPU
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = model.to(device)

# Freeze the feature extractor layers
for param in model.features.parameters():
    param.requires_grad = False

# Define transformations
transform_train = transforms.Compose([
    transforms.RandomHorizontalFlip(),
    transforms.RandomCrop(32, padding=4),
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

transform_test = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

# Load datasets
train_dataset = CIFAR10(root='./data', train=True, download=True, transform=transform_train)
test_dataset = CIFAR10(root='./data', train=False, download=True, transform=transform_test)

# Split the training dataset into training and validation sets
train_size = int(0.8 * len(train_dataset))
val_size = len(train_dataset) - train_size
train_dataset, val_dataset = random_split(train_dataset, [train_size, val_size])

# Create data loaders
train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True, num_workers=4)
val_loader = DataLoader(val_dataset, batch_size=64, shuffle=False, num_workers=4)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False, num_workers=4)
```

```
C:\Users\CherylPawluk\anaconda3\envs\pytorch2\lib\site-packages\torchvision\io\image.py:13: UserWarning: Failed to load image Python extension: '[WinError
127] The specified procedure could not be found'If you don't plan on using image functionality from `torchvision.io`, you can ignore this warning. Otherwis
e, there might be something wrong with your environment. Did you have `libjpeg` or `libpng` installed before building `torchvision` from source?
  warn(
C:\Users\CherylPawluk\anaconda3\envs\pytorch2\lib\site-packages\torchvision\models\_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated sin
ce 0.13 and may be removed in the future, please use 'weights' instead.
  warnings.warn(
C:\Users\CherylPawluk\anaconda3\envs\pytorch2\lib\site-packages\torchvision\models\_utils.py:223: UserWarning: Arguments other than a weight enum or `None`
for 'weights' are deprecated since 0.13 and may be removed in the future. The current behavior is equivalent to passing `weights=VGG16_Weights.IMAGENET1K_V
1`. You can also use `weights=VGG16_Weights.DEFAULT` to get the most up-to-date weights.
  warnings.warn(msg)
Files already downloaded and verified
Files already downloaded and verified
```

Step 2: Training

```
# Define the optimiser and learning rate scheduler
optimizer = optim.SGD(model.classifier.parameters(), lr=0.001, momentum=0.9)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=7, gamma=0.1)

# Define the loss function
criterion = nn.CrossEntropyLoss()

# Training Loop with early stopping to reduce training time
num_epochs = 5
best_val_loss = float('inf')
patience = 3
trigger_times = 0

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    for inputs, labels in train_loader:
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()

        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    model.eval()
    val_loss = 0.0
    with torch.no_grad():
        for inputs, labels in val_loader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            val_loss += loss.item()

    val_loss /= len(val_loader)

    if val_loss < best_val_loss:
        best_val_loss = val_loss
        trigger_times = 0
        torch.save(model.state_dict(), 'best_model.pth')
    else:
        trigger_times += 1
        if trigger_times >= patience:
            print(f"Early stopping at epoch {epoch+1}")
            break

print(f"Epoch {epoch+1}/{num_epochs}, Training Loss: {running_loss/len(train_loader)}, Validation Loss: {val_loss}")
scheduler.step()
```

Step 3: Data Loading and Checkpoints

```
# Load the best model
model.load_state_dict(torch.load('best_model.pth'))

# Replace this URL with the URL of an actual image from the CIFAR-10 dataset
url = "https://www.cs.toronto.edu/~kriz/cifar-10-sample/ship7.png"
response = requests.get(url)
img = Image.open(BytesIO(response.content))

# Convert to RGB if the image is in grayscale
if img.mode != 'RGB':
    img = img.convert('RGB')

# Apply the same transformations as used in training
transform_inference = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize((0.485, 0.456, 0.406), (0.229, 0.224, 0.225))
])

img_t = transform_inference(img)
img_t = img_t.unsqueeze(0).to(device) # Add batch dimension and move to GPU

# Inference
model.eval()
with torch.no_grad():
    output = model(img_t)
    _, predicted = torch.max(output, 1)
    predicted_class = predicted.item()

# CIFAR-10 class labels
class_labels = [
    'Airplane', 'Automobile', 'Bird', 'Cat', 'Deer', 'Dog',
    'Frog', 'Horse', 'Ship', 'Truck'
]

# Print the predicted class label
print(f'Predicted class: {class_labels[predicted_class]}')
```